Automate LLM Pipeline: Study Guide

Deploy TechCorp's Multi-Model Infrastructure — Module 1 hands-on practice

Overview

You are a DevOps engineer at TechCorp, a financial services company. Manual LLM deployments take **6 hours** and have caused **three production incidents** (about **$200K** in lost revenue). The task is to build an automated CI/CD pipeline that cuts deployment time to **under 30 minutes** with **zero-downtime** updates.

Learning objectives

#	Objective
LO1	Build optimized Docker containers for LLM applications using multi-stage builds and layer caching.
LO2	Create a GitHub Actions workflow for automated LLM container builds with security scanning.
LO3	Implement blue-green deployment strategies for zero-downtime model updates.
LO4	Define cloud infrastructure using Terraform for reproducible LLM deployments.
LO5	Compare automated pipeline performance against manual deployment benchmarks.

Tools and setup

- **Tools:** Docker Desktop or Engine, GitHub Actions (free tier is enough), Terraform CLI (1.0+), AWS CLI or another cloud CLI.
- **Accounts and hardware:** GitHub account with repo access; AWS/Azure/GCP free tier (or local Minikube); at least 8 GB RAM for container builds.
- **Starter files** (Module1_Deployment): `M1_app.py` (FastAPI app with health endpoints), `M1_Dockerfile.template`, `M1_requirements.txt`, `M1_main/variables/outputs.tf`, `M1_github_workflow.yml` (with TODOs).

Getting started

- Extract the starter zip and open the Module1_Deployment folder.
- Review `M1_app.py` and the Dockerfile template.
- Set up Docker, Terraform and the cloud CLI.
- Build incrementally from the templates and test each component before integrating.

Activity 1 — Create an optimized Dockerfile (LO1)

Step	What to do
1. Analyze the base image	Review the template and identify inefficiencies. Note which layers change often and which rarely. Check current size with `docker images`.
2. Multi-stage build	Builder stage on an NVIDIA CUDA base with Python; install build dependencies there. Runtime stage on a minimal CUDA runtime image; copy only the needed artifacts.
3. Layer caching	Order COPY from least to most frequently changing. Copy requirements.txt before app code. Make the model-weights download its own cached layer.
4. Build and verify	Build, compare size to the original, confirm the container starts and serves requests. Document the reduction (target: under 20 GB).

Issues in the template: single-stage build with build tools in the final image; `COPY . .` breaks caching; the full `-devel` CUDA image is large; no .dockerignore; runs as root. The template quotes a starting size of about 45 GB; the reference solution reports about 15 GB (a 67% reduction).

Tips: use .dockerignore; consider slim or Alpine variants where GPU support isn't needed; test caching by making a small code change and rebuilding.

Activity 2 — GitHub Actions workflow with security scanning (LO2, LO3)

Step	What to do
1. Workflow structure	Create `.github/workflows/deploy-llm.yml`. Trigger on push to main and on model-artifact changes. Set up environment variables for registry credentials.
2. Build + scan	Checkout and registry login; Docker layer caching; build and tag with the commit SHA; add a Trivy scan that fails on HIGH/CRITICAL.
3. Blue-green	Push the image to the registry; deploy with health-check validation; roll back automatically on failure; send a status notification.
4. Test the pipeline	Push a commit, watch the run in the Actions UI, verify the scan results and deployment, and record the execution time.

Tips: keep credentials in GitHub secrets; add timeouts to prevent runaway builds; consider parallel jobs for speed.

Activity 3 — Define infrastructure with Terraform (LO4, LO5)

Step	What to do
1. Module structure	Create main.tf, variables.tf, outputs.tf. Define the provider and a backend for state management.
2. Compute	GPU instance (g5.xlarge or equivalent) with the right AMI and user data; container registry (ECR/ACR/GCR).
3. Network and security	VPC with public and private subnets; security groups with the minimum required ports; Application Load Balancer.
4. Validate and compare	Run `terraform plan`; parameterize dev/staging/prod; document provisioning time vs. manual setup; compute total deployment time (build + scan + deploy + provision).

Tips: run `terraform fmt`; tag resources for cost tracking; consider modules for reusable components.

What to submit (peer review)

Compile a PDF report covering five requirements, then upload it to the Peer Review area with a short description and review your peers' work.

Requirement	Include
1. Optimized Docker container	Multi-stage Dockerfile; before/after size comparison; layer caching strategy; container health-check verification.
2. CI/CD with security scanning	Workflow YAML; Trivy configuration and results; evidence of a successful run; pipeline execution time.
3. Blue-green configuration	Blue-green config; health-check setup; automatic rollback trigger; evidence of zero downtime in a deployment test.
4. Infrastructure as code	main/variables/outputs.tf; terraform plan output; dev/staging/prod parameterization; tagging strategy.
5. Performance comparison	Deployment time (manual 6 h vs. automated); error rate; resource utilization; 3–5 sentence business analysis.

Key takeaways

- Multi-stage builds and layer caching can cut image size by roughly 50–70% and greatly improve build times (course figure).
- Automated CI/CD with security scanning turns 6-hour manual deployments into reliable ~30-minute workflows.
- Blue-green deployments with health checks enable zero-downtime updates and fast rollback.

Quick self-check

- Why copy requirements.txt before the application code?
- What does the runtime stage leave out compared with the builder?

- What does Trivy's exit-code 1 with HIGH/CRITICAL severity do to the pipeline?
- In blue-green, what must pass before traffic switches to green, and what happens on failure?